# Public-Key Encryption and Digital Signatures

CSE 405 January 2025
Lecture 7

# PRNGs: Summary



- True randomness requires sampling a physical process
  - Slow, expensive, and biased (low entropy)
- PRNG: An algorithm that uses a little bit of true randomness to generate a lot of random-looking output
  - Seed(entropy): Initialize internal state
  - Reseed(entropy): Add additional entropy to the internal state
  - Generate(n): Generate n bits of pseudorandom output
  - Security: Computationally indistinguishable from truly random bits
- HMAC-DRBG: Use repeated applications of HMAC to generate pseudorandom bits
- Application: UUIDs

# Summary: Diffie-Hellman Key Exchange



- Algorithm:
  - Alice chooses $a$ and sends $g^a \bmod p$ to Bob
  - Bob chooses $b$ and sends $g^b \bmod p$ to Alice
  - Their shared secret is $(g^a)^b = (g^b)^a = g^{ab} \bmod p$
- Diffie-Hellman provides forwards secrecy: Nothing is saved or can be recorded that can ever recover the key
- Diffie-Hellman can be performed over other mathematical groups, such as elliptic-curve Diffie-Hellman (ECDH)
- Issues
  - *Not* secure against MITM
  - Both parties must be online
  - Does not provide authenticity

# Diffie-Hellman: Security



Alice

**Mallory**

Public: $g$, $p$

Bob

Generate $a$

Calculate $g^a \bmod p$

Generate $m$

Calculate $g^m \bmod p$

Generate $b$

Calculate $g^b \bmod p$

Receive $g^m \bmod p$

Calculate $(g^m)^a \bmod p$

$a$, $g^a$, $g^m \Rightarrow g^{am}$

Receive $g^a \bmod p$

Calculate $(g^a)^m \bmod p$

$m$, $g^m$, $g^a \Rightarrow g^{am}$

Receive $g^b \bmod p$

Calculate $(g^b)^m \bmod p$

$m$, $g^m$, $g^b \Rightarrow g^{bm}$

Receive $g^m \bmod p$

Calculate $(g^m)^b \bmod p$

$b$, $g^b$, $g^m \Rightarrow g^{bm}$

# Public-Key Cryptography

# Public-Key Cryptography



- In public-key schemes, each person has two keys
  - **Public key**: Known to everybody
  - **Private key**: Only known by that person
  - Keys come in pairs: every public key corresponds to one private key
- Uses number theory
  - Examples: Modular arithmetic, factoring, discrete logarithm problem
  - Contrast with symmetric-key cryptography (uses XORs and bit-shifts)
- Messages are numbers
  - Contrast with symmetric-key cryptography (messages are bit strings)
- Benefit: No longer need to assume that Alice and Bob already share a secret
- Drawback: Much slower than symmetric-key cryptography
  - Number theory calculations are much slower than XORs and bit-shifts

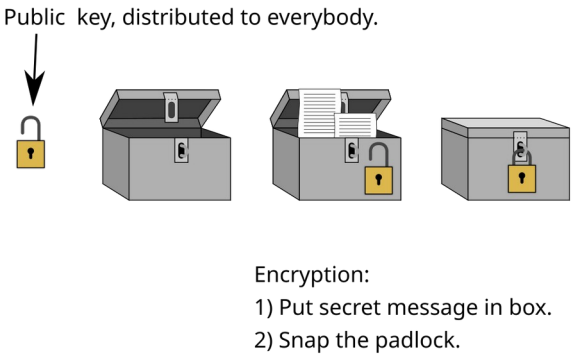
# Public-Key Encryption

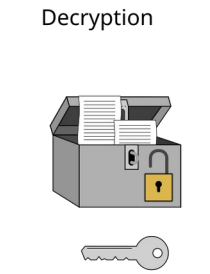
Textbook Chapter 11

# Public-Key Encryption



- Everybody can encrypt with the public key
- Only the recipient can decrypt with the private key



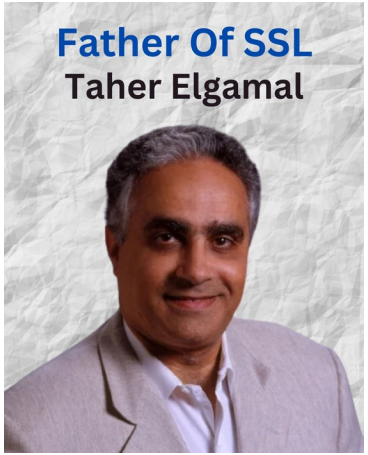




# Public-Key Encryption: Definition



- Three parts:
  - KeyGen() → *PK*, *SK*: Generate a public/private keypair, where *PK* is the public key, and *SK* is the private (secret) key
  - Enc(*PK*, *M*) → *C*: Encrypt a plaintext *M* using public key *PK* to produce ciphertext *C*
  - Dec(*SK*, *C*) → *M*: Decrypt a ciphertext *C* using secret key *SK*
- Properties
  - **Correctness**: Decrypting a ciphertext should result in the message that was originally encrypted
    - Dec(*SK*, Enc(*PK*, *M*)) = *M* for all *PK*, *SK* ← KeyGen() and *M*
  - **Efficiency**: Encryption/decryption should be reasonably fast

# ElGamal Encryption




Textbook Chapter 11.4

# Cryptography Roadmap



| | Symmetric-key | Asymmetric-key |
|---|---|---|
| Confidentiality | • One-time pads<br>• Block ciphers with chaining modes (e.g. AES-CBC) | • RSA encryption<br>• ElGamal encryption |
| Integrity, Authentication | • MACs (e.g. HMAC) | • Digital signatures (e.g. RSA signatures) |

- Hash functions
- Pseudorandom number generators
- Public key exchange (e.g. Diffie-Hellman)

- Key management (certificates)
- Password management

# ElGamal Encryption



- Diffie-Hellman key exchange is great: It lets Alice and Bob share a secret over an insecure channel
- Two problems:
  - Diffie-Hellman by itself can’t send messages. The secret $g^{ab} \bmod p$ is random.
  - Both Alice and Bob needs to be online in order to perform this key exchange.
- Idea: Let’s modify Diffie-Hellman so it supports encrypting and decrypting messages directly

# ElGamal Encryption: Protocol



- KeyGen():
  - Bob generates private key $b$ and public key $B = g^b \bmod p$
    - Intuition: Bob is completing his half of the Diffie-Hellman exchange
- Enc($B$, $M$):
  - Alice generates a random $r$ and computes $R = g^r \bmod p$
    - Intuition: Alice is completing her half of the Diffie-Hellman exchange
  - Alice computes $M \times B^r \bmod p$
    - Intuition: Alice derives the shared secret and multiples her message by the secret
  - Alice sends $C_1 = R$, $C_2 = M \times B^r \bmod p$
- Dec($b$, $C_1$, $C_2$)
  - Bob computes $C_2 \times C_1^{-b} = M \times B^r \times R^{-b} = M \times g^{br} \times g^{-br} = M \bmod p$
    - Intuition: Bob derives the (inverse) shared secret and multiples the ciphertext by the inverse shared secret

# ElGamal Encryption: Security



- Recall Diffie-Hellman problem: Given $g^a \bmod p$ and $g^b \bmod p$, hard to recover $g^{ab} \bmod p$
- ElGamal sends these values over the insecure channel
  - Bob's public key: $B$
  - Ciphertext: $R$, $M \times B^r \bmod p$
- Eve can't derive $g^{br}$, so she can't recover $M$

# ElGamal Encryption: Issues



- Is ElGamal encryption IND-CPA secure?
  - No. The adversary can send $M_0 = 0$, $M_1 \neq 0$
    - $C_1$ will be always 0, and $C_2$ will be always non-zero. Eve can easily distinguish.
  - Additional padding and other modifications are needed to make it semantically secure
- Malleability: The adversary can tamper with the message
  - The adversary can manipulate $C_1' = C_1$, $C_2' = 2 \times C_2 = 2 \times M \times g^{br}$ to make it look like $2 \times M$ was encrypted

# Exercise



In an ElGamal cryptosystem:

- The prime modulus is **11**.
- The primitive root (generator) is **2**.
- Bob's **private key** is **4**.
- Alice wants to send the message **7** to Bob using her **random key 3**.

Tasks:

- Compute Bob’s public key using the primitive root and his private key.
- Calculate the first and second part of the ciphertext. Show the complete ciphertext pair.
- Show how Bob decrypts the ciphertext.

# Exercise



- Mallory can’t read or replace Bob’s private key b, but she can multiply it by a known factor z. She wants to send a message M that decrypts as usual using the modified key b*z
  - What should Mallory send as C1?
  - What should Mallory send as C2?

Assume Bob’s public modulus B is unchanged.

# RSA Encryption

Textbook Chapter 11.3

# Cryptography Roadmap



| | Symmetric-key | Asymmetric-key |
|---|---|---|
| Confidentiality | • One-time pads<br>• Block ciphers with chaining modes (e.g. AES-CBC) | • RSA encryption<br>• ElGamal encryption |
| Integrity, Authentication | • MACs (e.g. HMAC) | • Digital signatures (e.g. RSA signatures) |

- Hash functions
- Pseudorandom number generators
- Public key exchange (e.g. Diffie-Hellman)

- Key management (certificates)
- Password management

# RSA Encryption: Definition



- KeyGen():
  - Randomly pick two large primes, $p$ and $q$
    - Efficient algorithm: pick random numbers and then use a test to see if the number is prime
  - Compute $N = pq$
    - $N$ is usually between 2048 bits and 4096 bits long
  - Choose $e$
    - Requirement: $e$ is relatively prime to $(p - 1)(q - 1)$
    - Requirement: $2 < e < (p - 1)(q - 1)$
  - Compute $d = e^{-1} \bmod (p - 1)(q - 1)$
    - Efficient algorithm for computing multiplicative inverses: Extended Euclid's algorithm
  - **Public key**: $N$ and $e$
  - **Private key:** $d$

# RSA Encryption: Definition



- From key generation:
  - $N = pq$
  - $d = e^{-1} \bmod (p - 1)(q - 1)$
  - Note: $(p - 1)(q - 1)$ can also be written as $\varphi(N)$
- Enc($e$, $N$, $M$):
  - Assume $M < N$ (If not, split M into blocks of size < N, otherwise you couldn't recover M)
  - $C = M^e \bmod N$
- Dec($d$, $C$):
  - $M = C^d \bmod N$
- For correctness, we need:

  Dec(d, Enc(e, N, M)) = $(M^e)^d = M \bmod N$

# RSA Encryption: Walkthrough



- Let p=5, q=11
  - Compute N and φ(N)
  - Choose a public key e = 3
  - Compute the corresponding private key d
  - Find the ciphertext for m=10
  - Decrypt the ciphertext and recover the original message

# Correctness of RSA Encryption



- Theorems: If $x = y \bmod p$, and $x = y \bmod q$, then $x = y \bmod pq$ (Chinese remainder theorem); $a^{p-1} = 1 \bmod p$ (Fermat's little theorem).
- From key generation, we know $d = e^{-1} \bmod (p - 1)(q - 1)$
  Therefore, $ed = 1 \bmod (p - 1)(q - 1)$
  Therefore, $ed - 1$ is a multiple of $(p - 1)(q - 1)$
- $M^{ed} = M^{ed-1}\, M \pmod p$
  $= M^{\text{some multiple of } p-1}\, M \pmod p$ (using definition of $ed - 1$)
  $= M^{(p-1)c}\, M \pmod p$
  $= M \pmod p$ (using FLT)
- Similarly $M^{ed} = M \pmod q$
- Therefore $M^{ed} = M \pmod N$
- Therefore Dec($d$, Enc($e$, $M$)) = $M$

# RSA Encryption: Security



- **RSA problem**: Given $N$ and $C = M^e \bmod N$, it is hard to find $M$
  - No harder than the factoring problem:
    If you can factor $N$, you can learn $p$ and $q$.
    Then, you can compute $d = e^{-1} \bmod (p - 1)(q - 1)$ and decrypt with $C^d \bmod N$.
- Current best solution is to factor $N$, but unknown whether there is an easier way
  - If the RSA problem is as hard as the factoring problem, then the scheme is secure as long as the factoring problem is hard
  - Factoring problem is assumed to be hard. Best known algorithms are exponential-time

# RSA Encryption: Issues



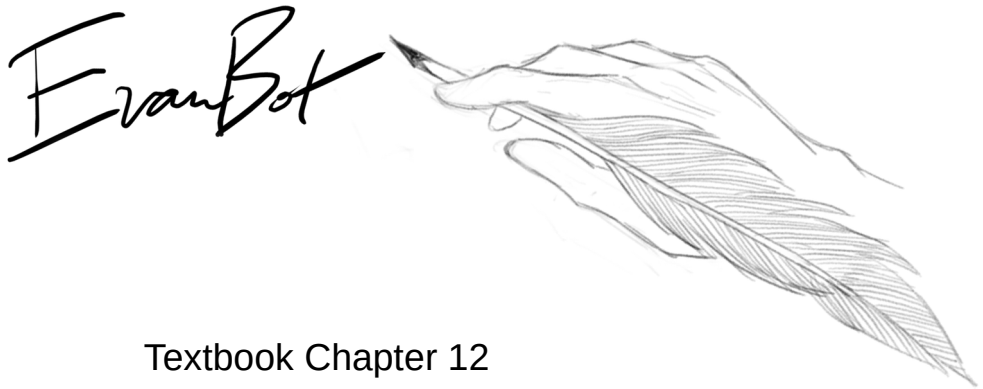
- Is RSA encryption IND-CPA secure?
  - No. It's deterministic. No randomness was used at any point!
- Sending the same message encrypted with different public keys also leaks information
  - $m^{e_a} \bmod N_a$, $m^{e_b} \bmod n_b$, $m^{e_c} \bmod N_c$
  - Small $m$ and $e$ leaks information
    - $e$ is usually small (~16 bits) and often constant (3, 17, 65537)
- Side channel: A poor implementation leaks information
  - The time it takes to decrypt a message depends on the message and the private key
  - This attack has been successfully used to break RSA encryption in OpenSSL
- Result: We need a probabilistic padding scheme

# Hybrid Encryption



- Issues with public-key encryption
  - Notice: We can only encrypt small messages because of the modulo operator
  - Notice: There is a lot of math, and computers are slow at math
  - Result: Asymmetric doesn't work for large messages
- **Hybrid encryption**: Encrypt data under a randomly generated key *K* using symmetric encryption, and encrypt *K* using asymmetric encryption
  - Benefit: Now we can encrypt large amounts of data quickly using symmetric encryption, and we still have the security of asymmetric encryption
- Almost all cryptographic systems use hybrid encryption

# Digital Signatures




Textbook Chapter 12

# Cryptography Roadmap



| | Symmetric-key | Asymmetric-key |
|---|---|---|
| Confidentiality | • One-time pads<br>• Block ciphers with chaining modes (e.g. AES-CBC) | • RSA encryption<br>• ElGamal encryption |
| Integrity, Authentication | • MACs (e.g. HMAC) | • Digital signatures (e.g. RSA signatures) |

- Hash functions
- Pseudorandom number generators
- Public key exchange (e.g. Diffie-Hellman)

- Key management (certificates)
- Password management

# Digital Signatures



- Asymmetric cryptography is good because we don’t need to share a secret key
- Digital signatures are the asymmetric way of providing integrity/authenticity to data
- Assume that Alice and Bob can communicate public keys without Mallory interfering
  - We will see how to fix this limitation later

# Public-key Signatures



- Only the owner of the private key can sign messages with the private key
- Everybody can verify the signature with the public key


# Digital Signatures: Definition



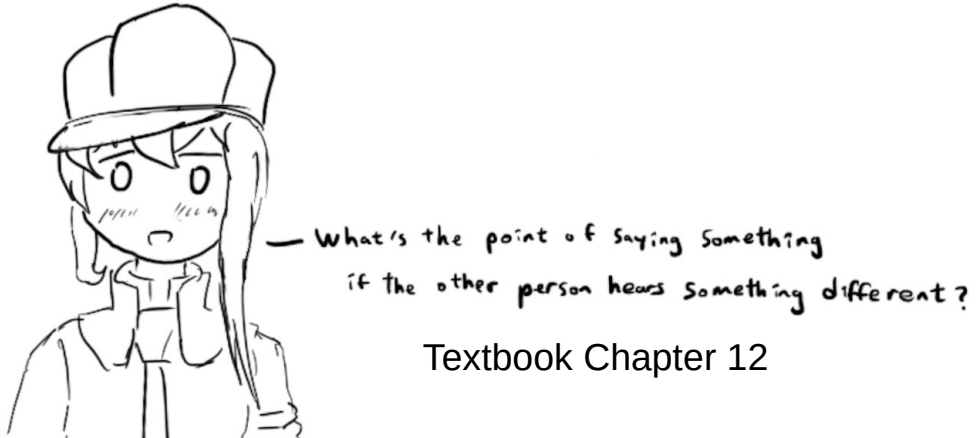
- Three parts:
  - KeyGen() → *PK*, *SK*: Generate a public/private keypair, where *PK* is the verify (public) key, and *SK* is the signing (secret) key
  - Sign(*SK*, *M*) → *sig*: Sign the message *M* using the signing key *SK* to produce the signature *sig*
  - Verify(*PK*, *M*, *sig*) → {0, 1}: Verify the signature *sig* on message *M* using the verify key *PK* and output 1 if valid and 0 if invalid
- Properties
  - **Correctness**: Verification should be successful for a signature generated over any message
    - Verify(*PK*, *M*, Sign(*SK*, *M*)) = 1 for all *PK*, *SK* ← KeyGen() and *M*
  - **Efficiency**: Signing/verifying should be fast
  - **Security**: EU-CPA, same as for MACs

# Digital Signatures in Practice



- If you want to sign message *M*:
  - First hash *M*
  - Then sign H(*M*)
- Why do digital signatures use a hash?
  - Allows signing arbitrarily long messages
- Digital signatures provide integrity *and authenticity* for *M*
  - The digital signature acts as proof that the private key holder signed H(*M*), so you know that *M* is authentically endorsed by the private key holder

# RSA Signatures




Textbook Chapter 12

# RSA Signatures



- Recall RSA encryption: $M^{ed} \equiv M \bmod N$
  - There is nothing special about using *e* first or using *d* first!
  - If we encrypt using *d*, then anyone can “decrypt” using *e*
    - Given *x* and $x^d \bmod N$, can’t recover *d* because of discrete-log problem, so *d* is safe

# RSA Signatures: Definition



- KeyGen():
  - Same as RSA encryption:
    - **Public key**: $N$ and $e$
    - **Private key:** $d$
- Sign($d$, $M$):
  - Compute $sig \equiv H(M)^d \bmod N$
- Verify($e$, $N$, $M$, $sig$)
  - Verify that $H(M) \equiv sig^e \bmod N$

# Quiz



- Suppose Alice wants to send a message M to Bob
- Can she send both the ciphertext (encrypted with RSA) and the signature of M (also signed with RSA) to Bob?
  - Think in terms of what the attacker can do

# DSA Signatures

# DSA Signatures



- A signature scheme based on Diffie-Hellman
  - The details of the algorithm are out of scope
- Usage
  - Alice generates a public-private key pair and publishes her public key
  - To sign a message, Alice generates a random, secret value *k* and does some computation
  - Note: *k* is not Alice's private key
  - Note: *k* is sometimes called a nonce but it is not: it must be *random* and never reused
  - The signature itself does not include *k*!
- *k* must be random and secret for each message
  - An attacker who learns *k* can also learn Alice's private key
  - If Alice reuses *k* on two signatures, an attacker can learn *k* (and use *k* to learn her private key)

# DSA Signatures: Attacks



- Sony PlayStation 3 (PS3)
- Digital rights management (DRM)
  - Prevent unauthorized code (e.g. pirated software) from running
  - The PS3 was designed to only run signed code
  - Signature algorithm: Elliptic-curve DSA
- Running alternate operating systems
  - The PS3 had an option to run alternate operating systems (Linux) that was later removed
  - This was catnip to reverse engineers (“The best way to get people interested is *removing* Linux from a device”)
- One of the authentication keys used to sign the firmware reused $k$ for multiple signatures → security lost!

# DSA Signatures: Attacks



- Android OS vulnerability (2013)
  - The "SecureRandom" function in its random number generator (RNG) wasn’t actually secure!
  - Not only was it low entropy, it would sometimes return the same value multiple times
- Multiple Bitcoin wallet apps on Android were affected
  - Bitcoin payments are signed with elliptic-curve DSA and published publicly
  - Insecure RNG caused multiple payments to be signed with the same *k*
- Attack: Someone scanned for all Bitcoin transactions signed insecurely
  - Recall: When multiple signatures use the same *k*, the attacker can learn *k* and the private key
  - In Bitcoin, your private key unlocks access to all your money

# DSA Signatures: Attacks



- Chromebooks have a built-in U2F (universal second factor) security key
  - Uses signatures to let the user log in to particular websites
  - Signature algorithm: 256-bit elliptic-curve DSA
- There was a bug in the secure hardware!
  - Instead of using 256-bit *k*, a bug caused *k* to be 32 bits long!
  - An attacker with a signature could simply try all possible values of *k*
- Fortunately the damage was slight
  - Each signature is only valid for logging into a single website
  - Each website used its own private key
- **Takeaway**: DSA (or ECDSA) is particularly vulnerable to incorrect implementations, compared with RSA signatures

# Summary: Public-Key Cryptography



- Public-key cryptography: Two keys; one undoes the other
- Public-key encryption: One key encrypts, the other decrypts
  - Security properties similar to symmetric encryption
  - ElGamal: Based on Diffie-Hellman
    - The public key is $g^b$, and $C_1$ is $g^r$.
    - Not IND-CPA secure on its own
  - RSA: Produce a pair $e$ and $d$ such that $M^{ed} = M \bmod N$
    - Not IND-CPA secure on its own
- Hybrid encryption: Encrypt a symmetric key, and use the symmetric key to encrypt the message
- Digital signatures: Integrity and authenticity for asymmetric schemes
  - RSA: Same as RSA encryption, but encrypt the hash with the *private* key